On-Device Training Library

Master Code Overview

All Files · Relationships · Function Reference

This document is a complete reference for the On-Device Training (ODT) C library. It covers every source file in the project, explains why each file exists, describes how files depend on each other, and documents every public function with its full syntax, parameter meanings, return value, and usage notes.

The project implements a lightweight neural-network training and inference engine designed to run on microcontrollers such as the STM32 Nucleo-F746ZG. It supports quantized tensors, arithmetic operations, convolutional and dense layers, loss functions, optimizers, serialization, and a clean user-facing API.

1. Project Architecture Overview

The library is organised into eight layers. Each layer builds on the layer below it. A file in layer N can only include files from layer N or lower — it never reaches up.

Layer	Files	Role
Layer 0 (Foundation)	Common.h DTypes.h / DTypes.c	Compiler macros (debug printing). Raw byte read/write helpers for int32 and float.
Layer 1 (Data types)	Quantization.h / Quantization.c	Enum of quantization schemes (INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL) and their config structs.
Layer 2 (Tensor)	Tensor.h / Tensor.c	Core tensor_t struct: data buffer, shape, quantization, sparsity. All shape/index/size calculations live here.
Layer 3 (Conversion)	TensorConversion.h / TensorConversion.c	Convert a tensor from one numeric type to another (e.g. float to quantized int). The conversionMatrix dispatch table.
Layer 4 (Arithmetic)	Arithmetic.h / Arithmetic.c Add, Sub, Mul, Div, Matmul, MinMax, Log, Square, Sum, Rounding, Comparison, ...	Element-wise and reduction operations on tensors. Arithmetic.h/.c provides the generic engine; each operation file (Add.h/.c, etc.) provides the concrete math function.
Layer 5 (Layers)	Layer.h / Layer.c Linear, Conv1d, Relu, Softmax, Dropout, Flatten, LayerNorm, AvgPool1d, MaxPool1d, ...	Neural-network layer implementations. Each layer has a forward pass (inference) and backward pass (gradient computation) built on top of Layer 4.
Layer 6 (Training)	LossFunction, CrossEntropy, MSE, Optimizer, Sgd, DataLoader, NPYLoader, RNG, Bernoulli	Loss computation, gradient-based weight update (SGD), dataset loading from .npy files, and random-number generation for Dropout.
Layer 7 (User API)	src/userApi/* InferenceApi, TrainingLoopApi, StateDictApi, StorageApi, TensorApi, QuantizationApi, ...	High-level entry points that a user calls to build, train, save, and run a model. Wraps everything below behind clean function calls.

Reading the dependency arrows

Every arrow in the diagram below points FROM a file TO the file it includes. If file A includes file B, then A depends on B — B must compile first, and any change to B's header forces A to recompile.

File	Includes / Depends On
Common.h	— (no project dependencies; only standard headers: stdio.h, string.h, stdbool.h)
DTypes.h / .c	— (no project dependencies; only stddef.h, stdint.h)
Quantization.h/.c	Rounding.h (for rounding-mode enum)
Tensor.h / .c	Common.h, DTypes.h, Quantization.h
TensorConversion.h/.c	Common.h, DTypes.h, Tensor.h, Quantization.h

Arithmetic.h / .c	Common.h, DTypes.h, Tensor.h, Matmul.h
Add.h / .c	Arithmetic.h, Tensor.h
Sub.h / .c	Arithmetic.h, Tensor.h
Mul.h / .c	Arithmetic.h, Tensor.h
Div.h / .c	Arithmetic.h, Tensor.h
Matmul.h / .c	Arithmetic.h, DTypes.h, Tensor.h
Layer.h / .c	Tensor.h, Arithmetic.h, TensorConversion.h
Linear.h / .c	Layer.h, Matmul.h, Add.h
Conv1d.h / .c	Layer.h, Conv1dKernel.h, Add.h
Relu.h / .c	Layer.h, Arithmetic.h
Softmax.h / .c	Layer.h, Arithmetic.h, Sum.h
LossFunction.h/.c	Tensor.h, Arithmetic.h
CrossEntropy.h/.c	LossFunction.h, Log.h, Sum.h
MSE.h / .c	LossFunction.h, Arithmetic.h
Optimizer.h / .c	Tensor.h, Arithmetic.h
Sgd.h / .c	Optimizer.h, Arithmetic.h
DataLoader.h / .c	Tensor.h, Dataset.h
NPYLoader.h / .c	DataLoader.h, DTypes.h
Serialize.h / .c	Tensor.h, DTypes.h
Deserialize.h/.c	Tensor.h, DTypes.h
RNG.h / .c	— (no project dependencies)
Bernoulli.h / .c	RNG.h, Tensor.h
src/userApi/*	All layers above

2. Common.h

PURPOSE

Common.h is the debug-printing infrastructure for the entire library. It defines three print macros (PRINT_DEBUG, PRINT_INFO, PRINT_ERROR) plus PRINT_BYTE_ARRAY. Every other .c file can include Common.h and immediately get coloured, level-controlled console output with the source file name and calling function name automatically included.

WHY THIS FILE EXISTS

On a microcontroller you cannot attach a debugger and step through code. The only feedback you get is what you print over UART. Having a single, uniform macro that prints the file name, function name, and message — and that can be silenced completely by not defining DEBUG_MODE_* — is therefore essential. Without Common.h, every .c file would need its own printf boilerplate.

DEPENDS ON

Standard library only: stdio.h, string.h, stdbool.h. No other project file.

USED BY

Every .c file in the project. It is the most widely included header in the codebase.

Macros defined in Common.h

```
#define DLEVEL 0 | 1 | 2 | 3
```

PURPOSE

Sets the global debug level. 0 = silent, 1 = errors only, 2 = info+errors, 3 = all. Determined at compile time by whether DEBUG_MODE_ERROR, DEBUG_MODE_INFO, or DEBUG_MODE_DEBUG is defined as a compiler flag (e.g. -DDEBUG_MODE_DEBUG).

PARAMETERS

No parameters — this is a compile-time constant.

NOTES

If none of the DEBUG_MODE_* flags are set, DLEVEL = 0 and ALL print macros produce zero code — the compiler optimises the entire do{...}while(false) away.

```
#define PRINT_DEBUG(str, ...)
```

PURPOSE

Prints a yellow-coloured debug message to stdout. Only active when DLEVEL >= 3 (i.e. -DDEBUG_MODE_DEBUG was passed to the compiler). The output format is: [SourceFile: FunctionName] your message

PARAMETERS

str: a printf-style format string literal. ...: optional extra arguments matching the format string.

NOTES

The SOURCE_FILE macro is defined by each .c file with #define SOURCE_FILE "MYFILE" before including Common.h. The __FUNCTION__ macro is a C99 built-in that expands to the name of the current function at compile time.

```
#define PRINT_INFO(str, ...)
```

PURPOSE

Prints a plain (no colour) informational message. Active when DLEVEL >= 2.

PARAMETERS

str: format string. ...: variadic arguments.

```
#define PRINT_ERROR(str, ...)
```

PURPOSE

Prints a RED-coloured error message. Active when DLEVEL >= 1. Used throughout the library to report illegal arguments, mismatched dimensions, and unsupported type combinations before calling exit(1).

PARAMETERS

str: format string. ...: variadic arguments.

NOTES

PRINT_ERROR is always paired with exit(1) in the project code. The macro does not call exit itself — the calling function does that.

```
#define PRINT_BYTE_ARRAY(prefix, byteArray, numberOfBytes)
```

PURPOSE

Dumps a raw byte array to stdout in hex format, e.g. 0x1A 0x2B 0x3C. Useful for inspecting the raw memory content of a tensor's data field.

PARAMETERS

prefix: a label string printed before the hex bytes. byteArray: pointer to uint8_t array. numberOfBytes: how many bytes to print.

3. DTypes.h / DTypes.c

PURPOSE

DTypes provides the bridge between the raw byte storage inside a tensor and the typed C values (int32_t, float) that arithmetic code needs to work with. A tensor stores all its data as a uint8_t byte array. Before you can add two tensor elements you must reassemble four bytes back into an int32_t or float. DTypes does exactly that — and nothing else.

WHY THIS FILE EXISTS

C does not let you safely cast a uint8_t* to an int32_t* — that would violate strict aliasing rules and produce undefined behaviour. The correct approach is memcpy, which DTypes wraps in named functions so the rest of the code stays readable. Having this in one place also means you change the byte order or packing format in exactly one file if hardware requires it.

DEPENDS ON

stddef.h (for size_t), stdint.h (for uint8_t, int32_t). No project dependencies.

USED BY

Tensor.c, TensorConversion.c, Arithmetic.c (and every file that does arithmetic or conversion), Serialize.c, Deserialize.c, NPYLoader.c.

Functions in DTypes.h / DTypes.c

```
int32_t readBytesAsInt32(uint8_t *bytes)
```

PURPOSE

Reads 4 bytes from the address 'bytes' and reassembles them into a signed 32-bit integer. Uses memcpy internally to avoid undefined behaviour.

PARAMETERS

bytes: pointer to the first of the 4 bytes to read. In practice this is always &tensor->data[byteIndex] where byteIndex is a multiple of 4.

RETURNS

Returns the int32_t value stored at that memory location.

NOTES

This is the single most-called function in the entire library. Every arithmetic operation on INT32 or SYM_INT32 tensors calls it once per element.

```
int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t numberOfBytes)
```

PURPOSE

Reads 'numberOfBytes' bytes (not necessarily 4) and sign-extends them into an int32_t. Used for tensors that pack data at less than 32 bits per element.

PARAMETERS

data: pointer to byte array. numberOfBytes: how many bytes to consume (1, 2, 3, or 4).

RETURNS

int32_t: the sign-extended integer value.

```
void readBytesAsInt32Array(size_t numberOfValues, uint8_t *bytes, int32_t *outputArray)
```

PURPOSE

Converts an entire byte buffer into an array of int32_t values by calling readBytesAsInt32 in a loop — one call per 4-byte group.

PARAMETERS

numberOfValues: number of int32_t values to extract. bytes: source byte array (must be at least numberOfValues*4 bytes long). outputArray: pre-allocated int32_t array that receives the results.

RETURNS

void (writes into outputArray).

```
float readBytesAsFloat(uint8_t *bytes)
```

PURPOSE

Reads 4 bytes and reassembles them as an IEEE 754 single-precision float. The companion of readBytesAsInt32 for FLOAT32 tensors.

PARAMETERS

bytes: pointer to 4 bytes in the tensor data buffer.

RETURNS

float: the floating-point value at that location.

NOTES

Like readBytesAsInt32, this uses memcpy to stay within the C standard.

```
void readBytesAsFloatArray(size_t numberOfValues, uint8_t *bytes, float *outputArray)
```

PURPOSE

Bulk-converts a byte buffer into a float array.

PARAMETERS

numberOfValues, bytes, outputArray — same pattern as readBytesAsInt32Array.

RETURNS

void (writes into outputArray).

```
void writeInt32ToByteArray(int32_t value, uint8_t *bytes)
```

PURPOSE

Writes a 32-bit signed integer into 4 consecutive bytes of a byte array. This is the inverse of readBytesAsInt32 and is called after every arithmetic result to store the answer back into a tensor's data field.

PARAMETERS

value: the int32_t result to store. bytes: pointer to 4 bytes in the output tensor's data buffer.

RETURNS

void — modifies the bytes in place.

NOTES

Always call this with &tensor->data[i*4] where i is the element index.

```
void writeInt32ArrayToByteArray(size_t numberOfValues, int32_t *valueArray, uint8_t *bytes)
```

PURPOSE
Bulk-writes an array of int32_t values into a byte buffer.

PARAMETERS
numberOfValues: number of values to write. valueArray: source int32_t array. bytes: destination byte buffer (must be at least numberOfValues*4 bytes).

RETURNS
void.

```
void writeFloatToByteArray(float value, uint8_t *bytes)
```

PURPOSE
Writes a float into 4 consecutive bytes. Inverse of readBytesAsFloat.

PARAMETERS
value: the float to store. bytes: destination in the tensor data buffer.

RETURNS
void.

```
void writeFloatArrayToByteArray(size_t numberOfValues, float *valueArray, uint8_t *bytes)
```

PURPOSE
Bulk float-to-bytes write.

PARAMETERS
numberOfValues, valueArray, bytes.

RETURNS
void.

4. Quantization.h / Quantization.c

PURPOSE

Quantization.h defines the type system for how tensors store numbers. On a microcontroller you cannot afford 32-bit floats everywhere. Quantization lets you represent values as small integers with a scale factor. This file defines: (a) the qtype_t enum naming the six supported schemes, (b) the config structs that store scale, zeroPoint, and bit-width for each scheme, and (c) init functions that fill those structs with safe default values.

WHY THIS FILE EXISTS

Without a uniform type system, every function would need to know whether a tensor holds floats, ints, or some quantized variant. The quantization_t struct bundles the scheme (qtype_t) with its parameters (qConfig void*) so that any function receiving a tensor_t can immediately inspect what kind of numbers are inside.

DEPENDS ON

Rounding.h (for roundingMode_t enum used in qConfig structs).

USED BY

Tensor.h (embedded in tensor_t), TensorConversion.h/.c, any layer that creates tensors.

Key types defined in Quantization.h

NOTE

qtype_t enum — six values: INT32 — raw 32-bit signed integer, no scale (used for intermediate accumulation) FLOAT32 — IEEE 754 single-precision float SYM_INT32 — symmetric quantization: real = mantissa * scale, centred at 0 SYM — symmetric with configurable bit-width (e.g. 8-bit) ASYM — asymmetric: real = (mantissa - zeroPoint) * scale BOOL — single-bit boolean stored in packed bytes quantization_t struct — two fields: qtype_t type — which scheme is being used void *qConfig — pointer to the matching config struct (symInt32QConfig_t, asymQConfig_t, etc.)

Functions in Quantization.h / Quantization.c

```
void initSymInt32QConfig(roundingMode_t roundingMode, symInt32QConfig_t *symInt32QConfig)
```

PURPOSE

Fills a symInt32QConfig_t with default values: scale = 1.0, qMaxBits = 16. The scale of 1.0 means no quantization effect initially — it is updated by requantSymInt32Tensor when the actual range is known.

PARAMETERS

roundingMode: how to round when quantizing (e.g. ROUND_NEAREST). symInt32QConfig: pointer to the struct to initialise.

RETURNS

void.

```
void initSymInt32QConfigWithQMaxBits(roundingMode_t roundingMode, symInt32QConfig_t
*symInt32QConfig, uint8_t qMaxBits)
```

PURPOSE

Same as initSymInt32QConfig but lets you choose a custom bit-width instead of the default 16. qMax = 2^(qMaxBits-1) - 1.

PARAMETERS

roundingMode, symInt32QConfig: same as above. qMaxBits: number of bits for the quantized range.

RETURNS

void.

```
void initSymQConfig(uint8_t qBits, roundingMode_t roundingMode, symQConfig_t *symQConfig)
```

PURPOSE

Initialises a symQConfig_t for symmetric quantization at a given bit-width.

PARAMETERS

qBits: total bits per element (e.g. 8). roundingMode. symQConfig: target struct.

RETURNS

void.

```
void initAsymQConfig(uint8_t qBits, roundingMode_t roundingMode, asymQConfig_t
*asymQConfig)
```

PURPOSE

Initialises an asymQConfig_t. Sets scale=1.0, zeroPoint=0 as defaults.

PARAMETERS

qBits, roundingMode, asymQConfig.

RETURNS

void.

```
void initInt32Quantization(quantization_t *quantization)
```

PURPOSE

Sets quantization->type = INT32 and quantization->qConfig = NULL. Use this when a tensor will store raw int32_t values with no scale factor.

PARAMETERS

quantization: pointer to the quantization_t to initialise.

RETURNS

void.

```
void initFloat32Quantization(quantization_t *quantization)
```

PURPOSE
Sets type = FLOAT32, qConfig = NULL.

PARAMETERS
quantization.

RETURNS
void.

```
void initBoolQuantization(quantization_t *quantization)
```

PURPOSE
Sets type = BOOL, qConfig = NULL.

PARAMETERS
quantization.

RETURNS
void.

```
void initSymInt32Quantization(symInt32QConfig_t *symInt32QConfig, quantization_t
*quantization)
```

PURPOSE
Sets type = SYM_INT32, qConfig = symInt32QConfig (cast to void*). After calling this, the quantization_t is fully wired: the type field tells code which scheme to use and the qConfig pointer leads to the scale and bit-width.

PARAMETERS
symInt32QConfig: pre-filled config struct. quantization: target wrapper struct.

RETURNS
void.

```
void initSymQuantization(symQConfig_t *symQConfig, quantization_t *quantization)
```

PURPOSE
Sets type = SYM, qConfig = symQConfig.

PARAMETERS
symQConfig, quantization.

RETURNS
void.

```
void initAsymQuantization(asymQConfig_t *asymQConfig, quantization_t *quantization)
```

PURPOSE
Sets type = ASYM, qConfig = asymQConfig.

PARAMETERS
asymQConfig, quantization.

RETURNS
void.

5. Tensor.h / Tensor.c

PURPOSE

Tensor.h defines the fundamental data structure of the entire library: tensor_t. A tensor is an N-dimensional array of numbers. tensor_t bundles together the raw byte buffer (data), the shape metadata (how many dimensions and how large each is), the quantization info (what kind of numbers are stored), and an optional sparsity descriptor. Tensor.c implements all the utility functions for working with that struct.

WHY THIS FILE EXISTS

Every layer, every operation, every loss function operates on tensor_t pointers. By keeping the data type generic (uint8_t*) and attaching quantization metadata, the same tensor struct can hold floats, ints, or quantized integers without changing any of the layer code. The layer just reads the quantization field to know how to interpret the bytes.

DEPENDS ON

Common.h, DTypes.h, Quantization.h, stdbool.h, stddef.h, stdint.h.

USED BY

Every single other file in the project. Tensor.h is the most important header.

NOTE

Key structs: shape_t { size_t numberOfDimensions; // how many axes (1=vector, 2=matrix, ...) size_t *dimensions; // size of each axis, in PHYSICAL storage order size_t *orderOfDimensions; // logical axis ordering (supports transpose) } tensor_t { uint8_t *data; // raw byte storage for all elements shape_t *shape; // dimension metadata quantization_t *quantization; // how bytes map to real values sparsity_t *sparsity; // optional sparsity structure (can be NULL) } parameter_t { tensor_t *param; // the weight tensor tensor_t *grad; // the gradient tensor (same shape as param) }

Functions in Tensor.h / Tensor.c

```
uint32_t getBitmask(uint32_t startbit, uint32_t endbit)
```

PURPOSE

Returns a 32-bit integer with bits set to 1 from startbit to endbit (inclusive). Used internally for bit-level tensor operations such as packing BOOL tensors.

PARAMETERS

startbit: lowest bit position (0 = LSB). endbit: highest bit position.

RETURNS

uint32_t bitmask.

```
uint8_t writeByte(uint8_t existingData, uint8_t data, uint8_t startbit, uint8_t endbit)
```

PURPOSE

Writes bits from 'data' into positions startbit..endbit of 'existingData', leaving all other bits untouched. Used for BOOL tensor packing.

PARAMETERS

existingData: the byte being modified. data: the value to write into the bit range. startbit / endbit: bit range within the byte.

RETURNS

uint8_t: the modified byte.

```
uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit)
```

PURPOSE

Extracts bits startbit..endbit from 'data' and returns them right-aligned.

PARAMETERS

data: the source byte. startbit / endbit: bit range to extract.

RETURNS

uint8_t: extracted bit field.

```
void byteConversion(uint8_t *dataIn, size_t dataInBits, uint8_t *dataOut, size_t
dataOutBits, size_t numValues)
```

PURPOSE

Converts between different per-element bit widths. For example, going from 1-bit (BOOL) to 8-bit storage or vice versa.

PARAMETERS

dataIn: source byte array. dataInBits: bits per element in source. dataOut: destination byte array. dataOutBits: bits per element in destination. numValues: how many logical values to convert.

RETURNS

void.

```
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex)
```

PURPOSE

Reads a single boolean value from a BOOL tensor at a given flat element index. Since booleans are stored packed (1 bit each), this extracts the correct bit from the correct byte.

PARAMETERS

tensor: pointer to a BOOL tensor. flatIndex: zero-based element index.

RETURNS

bool: true or false.

```
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value)
```

PURPOSE
Writes a boolean value into a BOOL tensor at flatIndex.

PARAMETERS
tensor: BOOL tensor. flatIndex: element index. value: true or false.

RETURNS
void.

```
tensor_t *getParamFromParameter(parameter_t *parameter)
```

PURPOSE
Returns the weight tensor pointer from a parameter_t.

PARAMETERS
parameter: pointer to a parameter_t.

RETURNS
tensor_t *param.

```
tensor_t *getGradFromParameter(parameter_t *parameter)
```

PURPOSE
Returns the gradient tensor pointer from a parameter_t.

PARAMETERS
parameter: pointer to a parameter_t.

RETURNS
tensor_t *grad.

```
size_t calcBytesPerElement(quantization_t *quantization)
```

PURPOSE
Returns how many bytes one element occupies, based on the quantization type. INT32 and FLOAT32 and SYM_INT32 return 4. BOOL returns 0 (sub-byte). SYM and ASYM return bytes based on qBits.

PARAMETERS
quantization: the quantization descriptor of a tensor.

RETURNS
size_t: bytes per element.

```
size_t calcBitsPerElement(quantization_t *quantization)
```

PURPOSE
Like calcBytesPerElement but returns bits. Needed for BOOL (1 bit/element).

PARAMETERS
quantization.

RETURNS
size_t: bits per element.

```
size_t calcBytesPerTensor(tensor_t *tensor)
```

PURPOSE
Returns the total number of bytes occupied by all elements in the tensor's data buffer. Equal to numberOfElements * bytesPerElement (rounded up for BOOL).

PARAMETERS
tensor: any tensor_t.

RETURNS
size_t: total byte count.

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements)
```

PURPOSE
Given a quantization scheme and element count, returns the byte count needed. Lower-level version of calcBytesPerTensor — used when you have the count before the tensor is fully built.

PARAMETERS
q: quantization descriptor. numberOfElements: how many values to store.

RETURNS
size_t: byte count.

```
size_t calcNumberOfElementsByShape(shape_t *shape)
```

PURPOSE
Multiplies all dimension sizes together to get the total element count. For a 3×4×5 tensor this returns 60.

PARAMETERS
shape: pointer to a shape_t.

RETURNS
size_t: total element count.

```
size_t calcNumberOfElementsByTensor(tensor_t *tensor)
```

PURPOSE
Convenience wrapper: calls calcNumberOfElementsByShape on tensor->shape.

PARAMETERS
tensor: any tensor_t.

RETURNS
size_t: total element count.

```
size_t calcNumberOfElementsByParameter(parameter_t *parameter)
```

PURPOSE
Returns element count of the param tensor inside the parameter_t.

PARAMETERS
parameter.

RETURNS
size_t.

```
void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index)
```

PURPOSE

Swaps two axes of a tensor by swapping their entries in orderOfDimensions. No data is moved — only the metadata changes. Any subsequent index calculation using calcElementIndexByIndices will automatically respect the new order.

PARAMETERS

tensor: the tensor to transpose. dim0Index: logical index of the first axis to swap. dim1Index: logical index of the second axis to swap.

RETURNS

void (modifies tensor->shape->orderOfDimensions in place).

NOTES

This is a zero-cost transpose — it is O(1) regardless of tensor size.

```
void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape, quantization_t
*quantization, sparsity_t *sparsity)
```

PURPOSE

Fills all four fields of a tensor_t at once. This is the standard way to initialise a tensor after the caller has already allocated all the sub-structs.

PARAMETERS

tensor: the tensor to fill. data: pre-allocated byte buffer. shape: pre-filled shape. quantization: pre-filled quantization. sparsity: sparsity descriptor or NULL.

RETURNS

void.

```
void setTensorValuesForConversion(uint8_t *data, quantization_t *q, tensor_t
*originalTensor, tensor_t *outputTensor)
```

PURPOSE

Sets up outputTensor to use the given data buffer and quantization while copying shape and sparsity from originalTensor. Used in TensorConversion.c when the output tensor needs the same shape but a different numeric type.

PARAMETERS

data: new byte buffer for the output. q: output quantization. originalTensor: source for shape/sparsity. outputTensor: tensor to configure.

RETURNS

void.

```
void setParameterValues(parameter_t *parameter, tensor_t *param, tensor_t *grad)
```

PURPOSE

Fills a parameter_t with its weight and gradient tensors.

PARAMETERS

parameter, param, grad.

RETURNS

void.

```
void setOrderOfDimsForNewTensor(size_t numberOfDimensions, size_t *orderOfDimensions)
```

PURPOSE
Initialises orderOfDimensions to the identity: [0, 1, 2, ..., N-1]. Call this when creating a new tensor that is not transposed.

PARAMETERS
numberOfDimensions: number of axes. orderOfDimensions: array to fill.

RETURNS
void.

```
void setShape(shape_t *shape, size_t *dims, size_t numberOfDims, size_t *orderOfDims)
```

PURPOSE
Fills all three fields of a shape_t at once.

PARAMETERS
shape: target struct. dims: dimension sizes. numberOfDims: rank. orderOfDims: ordering array.

RETURNS
void.

```
void printTensor(tensor_t *t)
```

PURPOSE
Prints tensor shape and raw byte data to stdout. Used for debugging.

PARAMETERS
t: any tensor.

RETURNS
void.

```
void printShape(shape_t *shape)
```

PURPOSE
Prints dimension count and each dimension size.

PARAMETERS
shape.

RETURNS
void.

```
void copyTensor(tensor_t *dest, tensor_t *src)
```

PURPOSE
Copies all fields of src into dest. This is a shallow copy of the struct fields (the pointers themselves are copied, not the data they point to).

PARAMETERS
dest: destination tensor. src: source tensor.

RETURNS
void.

6. TensorConversion.h / TensorConversion.c

PURPOSE

TensorConversion converts tensors from one numeric type to another. It answers the question: 'I have a FLOAT32 tensor; I need a SYM_INT32 tensor with the same values.' The conversion dispatch table (conversionMatrix) is a 6×6 grid of function pointers indexed by [fromType][toType]. The single public entry point convertTensor() looks up and calls the right converter.

WHY THIS FILE EXISTS

Neural-network training produces activations as floats but the hardware stores weights as quantized integers. You need a systematic way to move data between representations without writing special-case code everywhere. The matrix dispatch pattern means adding a new type requires filling in one new row and column, not touching existing code.

DEPENDS ON

Common.h, DTypes.h, Tensor.h, Quantization.h.

USED BY

Layer files that need to requantize activations, the user API QuantizationApi, and directly by training loop code.

The conversionMatrix — the dispatch table

conversionMatrix[6][6] is declared as extern in TensorConversion.h and defined in TensorConversion.c. Entry [i][j] holds a pointer to the function that converts from type i to type j. If a conversion is not supported, the entry holds a pointer to unsupportedConversionTypes() which prints an error and exits. The indices map to the qtype_t enum: 0=INT32, 1=FLOAT32, 2=SYM_INT32, 3=SYM, 4=ASYM, 5=BOOL.

NOTE

A _Static_assert at the bottom of TensorConversion.c checks that BOOL+1 == 6. If a new qtype_t value is ever added to the enum, this assertion will fail at compile time, reminding the developer to extend the matrix. This is a safety net built into the code.

Functions in TensorConversion.h / TensorConversion.c

```
typedef void (*conversionFunction_t)(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Defines the function-pointer type used in conversionMatrix. Every converter in the file has this exact signature: take the input tensor and the output tensor (which is already allocated), and fill the output with converted values.

PARAMETERS

inputTensor: the source tensor whose data field will be read. outputTensor: the destination tensor whose data field will be written.

RETURNS

void (the output tensor is modified in place).

NOTES

This is a typedef, not a function. It creates a new type name. You can declare a variable of this type: conversionFunction_t fn = someFunction;

```
void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

The main entry point for all type conversions. First checks if input and output have the same qtype_t — if so, calls convertTensorsWithSameType (memmove + copy metadata). Otherwise looks up conversionMatrix[inputType][outputType] and calls that function.

PARAMETERS

inputTensor: source tensor. outputTensor: pre-allocated output tensor with the desired quantization set.

RETURNS

void.

```
void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Re-quantizes a SYM_INT32 tensor into another SYM_INT32 tensor using a DYNAMICALLY computed scale. Two-pass algorithm: Pass A (read-only): scan all elements to find absMax = max|mantissa * inScale|. Compute fresh scale = absMax / qMax. Pass B: for each element, out = round(clamp(mantissa * inScale / scale, qMin, qMax)). Writes the new scale to outputTensor->quantization->qConfig->scale.

PARAMETERS

inputTensor: SYM_INT32 source. outputTensor: SYM_INT32 destination with qMaxBits set.

RETURNS

void.

NOTES

IN-PLACE SAFE: inputTensor == outputTensor is allowed because Pass A only reads and Pass B does a same-index read-then-write. The new scale is computed before any writes begin. This function is wired into conversionMatrix[SYM_INT32][SYM_INT32].

```
void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Like requantSymInt32Tensor but uses a PRE-SET target scale from outputTensor->quantization->qConfig->scale instead of computing a fresh one. Values that fall outside [qMin, qMax] are clamped (saturated). The caller must set the output scale before calling this function.

PARAMETERS

inputTensor: SYM_INT32 source. outputTensor: SYM_INT32 destination with scale already set.

RETURNS

void.

NOTES

SATURATES BY DESIGN: the clamping is intentional, not an error. NOT wired into conversionMatrix — must be called directly.

```
char *quantTypeToString(qtype_t t)
```

PURPOSE

Returns a human-readable string for a qtype_t value, e.g. "SYM_INT32". Used in error messages and debug output.

PARAMETERS

t: any qtype_t value.

RETURNS

char*: static string literal.

```
extern conversionFunction_t conversionMatrix[6][6]
```

PURPOSE

The dispatch table. Declared extern so other files can read it directly, e.g. to call conversionMatrix[SYM_INT32][FLOAT32](input, output).

PARAMETERS

N/A — this is a variable declaration, not a function.

RETURNS

N/A.

7. Arithmetic.h / Arithmetic.c

PURPOSE

Arithmetic.h / Arithmetic.c provides the generic engine for element-wise (point-wise) operations on tensors. Instead of implementing 'add', 'subtract', 'multiply' as separate loops, this file implements a SINGLE loop that takes a function pointer as a parameter. You pass in a function like 'int32 add(a, b)' and the loop calls it for every corresponding pair of elements. This pattern means all element-wise ops share one engine and any new operation is just one small function.

WHY THIS FILE EXISTS

Writing a separate loop for Add, Sub, Mul, Div, etc. would duplicate hundreds of lines of index calculation code. The function-pointer pattern keeps the loop logic in one place (Arithmetic.c) and keeps each concrete operation small (just the math, no looping). It also means index operations (including transpose-aware addressing) need to be correct in only one place.

DEPENDS ON

Common.h, DTypes.h, Tensor.h, Matmul.h (for calcNumberOfElementsByTensor).

USED BY

Add.h/.c, Sub.h/.c, Mul.h/.c, Div.h/.c, and all other operation files. Also Layer.h/.c.

NOTE

Two function-pointer typedefs: typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b) -- A function that takes two int32_t values and returns one int32_t. Example: int32_t addInts(int32_t a, int32_t b) { return a + b; } typedef float (*floatElementArithmeticFunc_t)(float a, float b) -- The float version of the same pattern. Four operation patterns: PointWise -- A op B -> output (two tensors in, one tensor out) PointWiseInplace -- A op B -> A (result overwrites A) ElementWithTensor -- scalar x op every element of tensor -> output ElementWithTensorInplace -- same but result overwrites tensor

Index helper functions

```
size_t getDimensionsByIndex(tensor_t *tensor, size_t index)
```

PURPOSE

Finds the logical dimension size for axis 'index'. Loops through orderOfDimensions[] to find which physical slot has logical index 'index', then returns dimensions[that slot]. This respects any transpose that was applied.

PARAMETERS

tensor: any tensor. index: logical axis number (0 = outermost / row axis).

RETURNS

size_t: size of that logical dimension. Exits with error if not found.

```
void orderDims(tensor_t *tensor, size_t *orderedDims)
```

PURPOSE

Fills orderedDims[0..N-1] with the logical dimension sizes in logical order. orderedDims[i] = getDimensionsByIndex(tensor, i). After this call, orderedDims is a transpose-aware snapshot of the shape.

PARAMETERS

tensor: any tensor. orderedDims: caller-allocated array of size numberOfDimensions.

RETURNS

void.

```
bool doDimensionsMatch(tensor_t *a, tensor_t *b)
```

PURPOSE

Checks that two tensors have the same rank and identical logical dimension sizes. First checks rank. Then calls orderDims on both and compares element by element. Exits with PRINT_ERROR if ranks differ.

PARAMETERS

a, b: two tensors to compare.

RETURNS

true if all dimensions match, false if any logical dimension differs.

NOTES

Must be called before any PointWise operation to guard against shape mismatches. All PointWise functions call this internally as their first step.

```
size_t calcTensorIndexByIndices(size_t numberOfDimensions, size_t *dimensions, size_t *indices)
```

PURPOSE

Converts a multi-dimensional index (e.g. [row=1, col=2]) into a flat integer index using row-major order. Formula (for 2D): flat = row * numCols + col. For N dimensions: starts from the last dimension and works backwards, accumulating an offset that grows by one dimension size per step.

PARAMETERS

numberOfDimensions: rank. dimensions: array of dimension sizes. indices: array of per-axis positions.

RETURNS

size_t: the flat element index.

NOTES

Example: tensor shape [3,4], indices [1,2] → flat = 1*4 + 2 = 6.

```
void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims, size_t rawIndex, size_t *indices)
```

PURPOSE

The INVERSE of calcTensorIndexByIndices. Given a flat index, recovers the per-axis indices. First computes total elements as the product of all dims. Then peels off one axis at a time from the outermost: indices[i] = rawIndex / (total / dims[i]), then reduces rawIndex by that amount.

PARAMETERS

numberOfDims: rank. dims: dimension sizes. rawIndex: the flat position. indices: array to fill.

RETURNS

void (fills the indices array).

NOTES

Example: dims=[3,4], rawIndex=6 → total=12, peel dim0: 6/4=1, rest=6-4=2; peel dim1: 2/1=2 → [1,2].

```
size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims, size_t *indices, size_t *orderOfDimensions)
```

PURPOSE

TRANSPOSE-AWARE flat index. First remaps the logical indices to physical order using orderOfDimensions, then applies the same row-major formula as calcTensorIndexByIndices. This is the key function that makes transposed tensors work correctly: the same logical element appears at a different physical byte position depending on the transpose state.

PARAMETERS

numberOfDims, dims: shape info. indices: logical per-axis positions. orderOfDimensions: the orderOfDimensions array from the tensor's shape.

RETURNS

size_t: the physical flat index into the data buffer.

NOTES

Used inside PointWise loops so that tensor A and tensor B can have different transpose states and still be combined correctly element-by-element.

PointWise arithmetic functions (two tensors)

```
void int32PointWiseArithmetic( tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE

The core element-wise loop for int32 tensors, writing to a SEPARATE output tensor. For each flat position i: decode i → logical indices in A → physical byte offset in A; repeat for B; read both values; call arithmeticFunc(a, b); write result sequentially to output.

PARAMETERS

aTensor: first operand (INT32 or SYM_INT32). bTensor: second operand. arithmeticFunc: the operation to apply (add, subtract, multiply, etc.). outputTensor: pre-allocated result tensor.

RETURNS

void.

NOTES

The output is always written SEQUENTIALLY (i * bytesPerElement) even if A and B are transposed. The output tensor gets a non-transposed layout.

```
void int32PointWiseArithmeticInplace( tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE

Same logic as int32PointWiseArithmetic but writes the result back into aTensor->data instead of a separate output. No output tensor parameter.

PARAMETERS

aTensor: first operand; also receives the result. bTensor: second operand. arithmeticFunc: operation to apply.

RETURNS

void (aTensor is modified in place).

NOTES

Use this when you no longer need the original aTensor values after the operation.

```
void floatPointWiseArithmetic( tensor_t *aTensor, tensor_t *bTensor,
floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE

Identical to int32PointWiseArithmetic but for FLOAT32 tensors. Uses readBytesAsFloat / writeFloatToByteArray instead of the int32 variants.

PARAMETERS

aTensor, bTensor: FLOAT32 tensors. arithmeticFunc: float operation. outputTensor: FLOAT32 result.

RETURNS

void.

```
void floatPointWiseArithmeticInplace( tensor_t *aTensor, tensor_t *bTensor,
floatElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE
Float in-place version. Result overwrites aTensor.

PARAMETERS
aTensor, bTensor: FLOAT32 tensors. arithmeticFunc: float operation.

RETURNS
void.

ElementWithTensor functions (scalar × tensor)

```
void int32ElementWithTensorArithmetic( tensor_t *aTensor, int32_t x,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE
Applies arithmeticFunc(element, x) to every element of aTensor, writing to outputTensor. Simpler than PointWise — no dimension check needed and the loop is purely sequential because the scalar x is the same for every element.

PARAMETERS
aTensor: input tensor. x: the scalar operand. arithmeticFunc: operation (e.g. multiply all elements by x). outputTensor: result.

RETURNS
void.

```
void int32ElementWithTensorArithmeticInplace( tensor_t *aTensor, int32_t x,
int32ElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE
Same but writes the result back to aTensor.

PARAMETERS
aTensor: input and output. x: scalar. arithmeticFunc: operation.

RETURNS
void.

```
void floatElementWithTensorArithmetic( tensor_t *aTensor, float x,
floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE
Float version: applies arithmeticFunc(element, x) to every float element, writing to output.

PARAMETERS
aTensor: FLOAT32 input. x: float scalar. arithmeticFunc. outputTensor: FLOAT32 result.

RETURNS
void.

```
void floatElementWithTensorArithmeticInplace( tensor_t *aTensor, float x,
floatElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE

Float scalar in-place version.

PARAMETERS

aTensor: FLOAT32 input and output. x: scalar. arithmeticFunc.

RETURNS

void.

8. Higher-Level Files — Summary

The files described below are built on top of the foundation files (Sections 2–7). They use the same patterns — tensor_t, function pointers, quantization types — but focus on specific neural-network operations rather than generic primitives.

8.1 Specialised Arithmetic Files (src/arithmetic/)

Each file implements one concrete operation by providing a function that matches int32ElementArithmeticFunc_t or floatElementArithmeticFunc_t and passing it to the generic engine in Arithmetic.c. Add.c contains int32 add(a,b){return a+b;} and a wrapper that calls int32PointWiseArithmetic with that function. Sub, Mul, Div follow the same pattern. Matmul.c implements matrix multiplication using its own loop (not the element-wise engine). MinMax.c finds minimum or maximum. Log.c computes natural logarithm. Square.c squares every element. Sum.c sums all elements into a scalar. Rounding.c provides the round/floor/ceil modes used by quantization. Comparison.c does element-wise greater-than, less-than, equal. Distributions.c implements softmax probability distribution. Kernel.c, SlidingWindow1d.c, AdaptiveWindow1d.c, Conv1dKernel.c, ConvTranspose1dKernel.c implement the inner loops of convolution operations.

8.2 Neural-Network Layer Files (src/layer/)

Layer.h / Layer.c defines the base layer struct and forward/backward function pointer types. Each specific layer (Linear, Conv1d, Relu, Softmax, Dropout, Flatten, LayerNorm, AvgPool1d, MaxPool1d, AdaptiveAvgPool1d, QuantizationLayer) implements the forward pass (inference: compute output from input) and the backward pass (training: compute gradients with respect to weights and inputs). A Linear layer computes output = input × weight + bias using Matmul.c and Add.c. A Conv1d layer uses Conv1dKernel.c to slide a filter over the input. Relu sets negative values to zero using Comparison.c and Arithmetic.c. Softmax calls Distributions.c. QuantizationLayer calls TensorConversion.c to quantize activations between layers.

8.3 Training Infrastructure

LossFunction.h / .c is the base for loss computation. CrossEntropy.h/.c implements the cross-entropy loss used for classification. MSE.h/.c implements mean-squared error. Both compute a scalar loss value and its gradient with respect to the layer output. Optimizer.h / .c is the base for weight-update rules. Sgd.h/.c implements stochastic gradient descent: for each parameter, weight -= learningRate * gradient. DataLoader.h / .c provides a dataset iterator. NPYLoader.h/.c reads NumPy .npy binary files from storage and fills tensor_t structs, enabling the model to be trained from real data files saved from Python.

8.4 Serialization (src/serial/)

Serialize.h/.c writes a model's weight tensors to a binary file in flash or SD card memory. Deserialize.h/.c reads them back. These files make it possible to save a trained model once and reload it on subsequent boots without re-training.

8.5 RNG (src/rng/)

RNG.h/.c provides a random-number generator. Bernoulli.h/.c generates a binary mask tensor where each entry is independently 1 with probability p and 0 with probability 1-p. This mask is used by Dropout to randomly zero out activations during training.

8.6 User API (src/userApi/)

The userApi folder contains one file per domain. InferenceApi.h provides functions to run a forward pass through the full model. TrainingLoopApi.h provides a train-one-epoch loop. StateDictApi.h provides load/save for named model parameters. StorageApi.h abstracts flash or file-system storage. TensorApi.h wraps tensor creation and access for the user. QuantizationApi.h wraps scale initialisation. LayerWeightsApi.h gives access to weight tensors by layer index. ModelValidationApi.h provides validation-set evaluation. Each layer has its own Api file (LinearApi, Conv1dApi, ReluApi, etc.) that lets the user configure and connect that layer without touching internal structs.

9. Quick Function Reference Card

The table below lists every public function covered in this document with its file and a one-line description. Use it as an index when you are looking for a specific function.

Function / Macro	File	One-line description
PRINT_DEBUG(str,...)	Common.h	Print yellow debug message (DLEVEL>=3)
PRINT_INFO(str,...)	Common.h	Print info message (DLEVEL>=2)
PRINT_ERROR(str,...)	Common.h	Print red error message (DLEVEL>=1)
PRINT_BYTE_ARRAY(p,a,n)	Common.h	Dump byte array as hex
readBytesAsInt32	DTypes.h	4 bytes → int32_t
readNumberOfBytesAsInt32	DTypes.h	N bytes → int32_t (sign-extended)
readBytesAsInt32Array	DTypes.h	Byte buffer → int32_t array
readBytesAsFloat	DTypes.h	4 bytes → float
readBytesAsFloatArray	DTypes.h	Byte buffer → float array
writeInt32ToByteArray	DTypes.h	int32_t → 4 bytes
writeInt32ArrayToByteArray	DTypes.h	int32_t array → byte buffer
writeFloatToByteArray	DTypes.h	float → 4 bytes
writeFloatArrayToByteArray	DTypes.h	float array → byte buffer
initSymInt32QConfig	Quantization.h	Fill symInt32QConfig (default 16-bit)
initSymInt32QConfigWithQMaxBits	Quantization.h	Fill symInt32QConfig (custom bits)
initSymQConfig	Quantization.h	Fill symQConfig
initAsymQConfig	Quantization.h	Fill asymQConfig
initInt32Quantization	Quantization.h	Set type=INT32
initFloat32Quantization	Quantization.h	Set type=FLOAT32
initBoolQuantization	Quantization.h	Set type=BOOL
initSymInt32Quantization	Quantization.h	Set type=SYM_INT32 + link config
initSymQuantization	Quantization.h	Set type=SYM + link config
initAsymQuantization	Quantization.h	Set type=ASYM + link config
getBitmask	Tensor.h	Build a bit mask for startbit..endbit
writeByte	Tensor.h	Write bits into a byte
readByte	Tensor.h	Extract bits from a byte
byteConversion	Tensor.h	Convert between different bit-widths
tensorBoolGet	Tensor.h	Read one bool from a BOOL tensor
tensorBoolSet	Tensor.h	Write one bool to a BOOL tensor
getParamFromParameter	Tensor.h	Get weight tensor from parameter_t
getGradFromParameter	Tensor.h	Get gradient tensor from parameter_t
calcBytesPerElement	Tensor.h	Bytes per element for a given qtype
calcBitsPerElement	Tensor.h	Bits per element for a given qtype

calcBytesPerTensor	Tensor.h	Total bytes in tensor data buffer
calcNumberOfBytesForData	Tensor.h	Bytes needed for N elements of given type
calcNumberOfElementsByShape	Tensor.h	Product of all dimension sizes
calcNumberOfElementsByTensor	Tensor.h	Elements in a tensor
calcNumberOfElementsByParameter	Tensor.h	Elements in parameter's weight
transposeTensor	Tensor.h	Swap two axes (zero-cost metadata change)
setTensorValues	Tensor.h	Fill all four tensor_t fields at once
setTensorValuesForConversion	Tensor.h	Set data+quant, copy shape from other
setParameterValues	Tensor.h	Fill a parameter_t
setOrderOfDimsForNewTensor	Tensor.h	Init orderOfDimensions to identity
setShape	Tensor.h	Fill a shape_t
printTensor	Tensor.h	Debug-print tensor to stdout
printShape	Tensor.h	Debug-print shape to stdout
copyTensor	Tensor.h	Shallow copy of tensor_t fields
convertTensor	TensorConversion.h	Dispatch conversion via matrix
requantSymInt32Tensor	TensorConversion.h	SYM_INT32 → SYM_INT32 dynamic scale
requantSymInt32TensorToScale	TensorConversion.h	SYM_INT32 → SYM_INT32 fixed scale
quantTypeToString	TensorConversion.h	qtype_t → readable string
conversionMatrix[6][6]	TensorConversion.h	Dispatch table of converters
getDimensionsByIndex	Arithmetic.h	Logical dimension size for an axis
orderDims	Arithmetic.h	Fill array with logical dim sizes
doDimensionsMatch	Arithmetic.h	Check two tensors have same shape
calcTensorIndexByIndices	Arithmetic.h	Multi-dim indices → flat index
calcIndicesByRawIndex	Arithmetic.h	Flat index → multi-dim indices
calcElementIndexByIndices	Arithmetic.h	Transpose-aware flat index
int32PointWiseArithmetic	Arithmetic.h	Element-wise int32 A op B → output
int32PointWiseArithmeticInplace	Arithmetic.h	Element-wise int32 A op B → A
floatPointWiseArithmetic	Arithmetic.h	Element-wise float A op B → output
floatPointWiseArithmeticInplace	Arithmetic.h	Element-wise float A op B → A
int32ElementWithTensorArithmetic	Arithmetic.h	int32 scalar op every element → out
int32ElementWithTensorArithmeticInplace	Arithmetic.h	int32 scalar op tensor → tensor
floatElementWithTensorArithmetic	Arithmetic.h	float scalar op every element → out
floatElementWithTensorArithmeticInplace	Arithmetic.h	float scalar op tensor → tensor